

Problem Set 4

Important:

- Write your name as well as your NU ID on your assignment. Please number your problems.
- Submit both results and your code.
- Give complete answers. Do not just give the final answer; instead show steps you went through to get there and explain what you are doing. Do not leave out critical intermediate steps.
- This assignment must be submitted electronically through Gradescope by December 11th 2024 (Wednesday) by 11:59 PM.

1 Generative Adversarial Network (GAN)

In this problem, we will implement a generative adversarial network (GAN) that models a high-dimensional data distribution $p_{data}(\vec{x})$, where $\vec{x} \in \mathbb{R}^d$. We define a generator network G_θ which produces samples $G_\theta(\vec{z})$ by sampling a k -dimensional random vector $\vec{z} \sim \mathcal{N}(\vec{0}, I)$. We also define a discriminator D_ϕ which judges how realistic the generated images $G_\theta(\vec{z})$ are in comparison to samples from the data distribution $p_{data}(\vec{x})$. As discussed in class, given that $D_\phi(\vec{x})$ is supposed to be interpreted as a probability that $\vec{x}$ was generated from the data distribution, the last layer of the discriminator is given by the sigmoid function,

$$\sigma(x) = \frac{1}{1 + e^{-x}},$$

which constrains its output to fall between 0 and 1. The discriminator's logits, $h_\phi((\vec{x}))$, denote the values of the discriminator right before the sigmoid layer, i.e. $D_\phi(\vec{x}) = \sigma(h_\phi((\vec{x})))$. Larger values of $h_\phi((\vec{x}))$ indicate a higher probability that the input to the discriminator was sampled from $p_{data}(\vec{x})$.

There are several choices of loss functions that can be used to train GANs. In class, we defined the discriminator's loss $L_D(\phi, \theta)$ as,

$$L_D(\phi, \theta) = -E_{\vec{x} \sim p_{data}(\vec{x})} [\log D_\phi(\vec{x})] - E_{\vec{z} \sim \mathcal{N}(\vec{0}, I)} \left[\log \left(1 - D_\phi(G_\theta(\vec{z})) \right) \right],$$

and the generator's loss $L_G(\theta, \phi)$ as,

$$L_G(\theta, \phi) = E_{\vec{z} \sim \mathcal{N}(\vec{0}, I)} \left[\log \left(1 - D_\phi(G_\theta(\vec{z})) \right) \right] = E_{\vec{z} \sim \mathcal{N}(\vec{0}, I)} \left[\log \left(1 - \sigma(h_\phi((G_\theta(\vec{z})))) \right) \right].$$

1. We will show that the generator's loss $L_G(\theta, \phi)$ suffers from a vanishing gradient problem. Show that derivative of $L_G(\theta, \phi)$ with respect to θ is given by,

$$\frac{\partial L_G}{\partial \theta} = -E_{\vec{z} \sim \mathcal{N}(\vec{0}, I)} \left[D_\phi(G_\theta(\vec{z})) \cdot \frac{\partial}{\partial \theta} h_\phi((G_\theta(\vec{z}))) \right].$$

You can use the fact that $\sigma'(x) = \sigma(x)(1 - \sigma(x))$.

2. In class, we said that when the discriminator successfully identifies a fake sample $G_\theta(\vec{z})$, we have $D_\phi(G_\theta(\vec{z})) \approx 0$. What happens to the training of the generator in this scenario?

Hint: Write out the gradient descent update rule for the generator and compute the value of $\frac{\partial L_G}{\partial \theta}$ when $D_\phi(G_\theta(\vec{z})) \approx 0$.

3. Given vanishing gradient problem, in practice, another loss function, known as the non-saturating loss, is used to train the generator which reads,

$$L_G^{non-sat}(\theta, \phi) = -E_{\vec{z} \sim \mathcal{N}(\vec{0}, I)} \left[\log \left(D_\phi(G_\theta(\vec{z})) \right) \right].$$

Explain why we don't have the vanishing gradient issue anymore with the non-saturating loss function.

4. To train a GAN model using the loss functions, we approximate $L_D(\phi, \theta)$ and $L_G^{non-sat}(\theta, \phi)$ using Monte Carlo sampling resulting in,

$$L_D(\phi, \theta) \approx -\frac{1}{n} \sum_{i=1}^n \log D_\phi(\vec{x}^{(i)}) - \frac{1}{n} \sum_{i=1}^n \log \left(1 - D_\phi(G_\theta(\vec{z}^{(i)})) \right),$$

$$L_G^{non-sat}(\theta, \phi) \approx -\frac{1}{n} \sum_{i=1}^n \log \left(D_\phi(G_\theta(\vec{z}^{(i)})) \right),$$

where n is the batch size, and for $i = 1, \dots, n$, we sample $\vec{x}^{(i)} \sim p_{data}(\vec{x})$ and $\vec{z}^{(i)} \sim \mathcal{N}(\vec{0}, I)$.

Implement and train a non-saturating GAN on Fashion MNIST for one epoch. Read through `run_gan.py`, and in `codebase/gan.py`, implement the `loss_nonsaturating` function. To train the model, execute `python run_gan.py`. You may monitor the GAN's output in the `out_nonsaturating` directory. Note that because the GAN is only trained for one epoch, we cannot expect the model's output to produce very realistic samples, but they should be roughly recognizable as clothing items.